

Unlearning at a Distance: Moving Far in Parameter Space

Jebastin Nadar Bobbi Halsell Yixuan (Lynn) Liu Varsha Bubathi
Aleksandra (Ola) Pasieka Andrew Siow
{jebastin.nadar24, bobbi.halsell24, yixuan.liu118, varsha.bubathi24,
aleksandra.pasieka24, andrew.siow24}@imperial.ac.uk

Supervisors: Dr Hamed Haddadi, Mohammad Maheri

COMP70079
Department of Computing
Imperial College London

September 22nd, 2025

Contents

1	Introduction	1
1.1	Machine unlearning	1
1.2	Machine unlearning notation	1
1.3	Evaluating Machine Unlearning methods	1
1.4	Motivation	1
1.5	Existing work	2
2	Logistics	2
2.1	Team management	2
2.2	Software management	3
3	Technical Decisions, Design, and Architecture	3
3.1	SafeUnlearning overview	3
3.2	Dataset app	4
3.3	Train app	5
3.4	Unlearning app	6
3.5	Evaluation app	7
3.6	Attacks app	7
4	Developing a New Machine Unlearning Method	8
4.1	Description	8
4.2	Subspace Gradient Redirection Unlearning	8
5	Adapting Reconstruction Attacks from Federated Learning	9
5.1	Description	9
5.2	Implementing reconstruction attacks	9
5.2.1	Evaluation metrics	9
5.2.2	Attack assumptions	9
5.3	Generative Gradient Leakage	9
5.3.1	Implementation details	9
5.3.2	Results	10
5.4	Inverting gradients	10
5.4.1	Implementation details	11
5.4.2	Results	11
6	Adapting Membership Inference Attacks	12
6.1	Likelihood Ratio Attack	12
6.1.1	Adapting LiRA for unlearning	12
6.1.2	Implementing U-LiRA	12
6.1.3	Extending U-LiRA	13
6.1.4	Results	13
6.2	Gradient Likelihood Ratio (GLiR) attack	13
6.2.1	Adapting GLiR for unlearning	14
6.2.2	Implementing GLiR	14
6.2.3	GLiR Results	14
7	Conclusion	15
7.1	Research milestones	15
7.2	Future work	15

1 Introduction

1.1 Machine unlearning

Artificial intelligence is a powerful tool that is transforming how we process information across various domains. While increasing compute, memory, and data volumes improve model performance and accuracy [1], the growing need for large, high-quality datasets has raised privacy concerns, prompting stricter regulations such as the “right to be forgotten” [2], where individuals can request the removal of their personal information from the training data. Beyond privacy, the need to remove data also arises when a model has been trained on poisoned or problematic data. As such, machine unlearning aims to remove the influence of specific samples from trained models, and is generally classified into two approaches: exact and approximate unlearning [3]. While theoretically the best approach would be exact unlearning, which involves retraining the model without the unwanted samples, this is typically infeasible due to the size of modern models and the cost incurred. Furthermore, sequential removal requests would require multiple complete retraining cycles, making this approach impractical for real-world applications. Consequently, approximate unlearning methods have gained considerable attention in recent years as a faster and more tractable alternative.

1.2 Machine unlearning notation

Given a dataset $\mathcal{D}$ sampled from some underlying distribution $\mathbb{D}$, let θ_o represent the neural network trained on $\mathcal{D}$. The set of data points chosen for unlearning is defined as the forget set $\mathcal{D}_f \subset \mathcal{D}$. Its complement $\mathcal{D}_r = \mathcal{D} \setminus \mathcal{D}_f$ is defined as the retain set. The goal of approximate machine unlearning is to generate an unlearned model θ_u from θ_o that effectively approximates θ_r : the model trained on $\mathcal{D}_r$.

1.3 Evaluating Machine Unlearning methods

We assess unlearned models based on two key aspects: model utility, the ability of θ_u to preserve the performance of θ_o on $\mathcal{D}_r$ and some test set $\mathcal{D}_{test} \subset \mathbb{D}$; and forget quality, the extent to which performance on $\mathcal{D}_f$ is reduced. While they are important measures for evaluating machine unlearning, they do not consider the extent to which the forget data can still be inferred after the unlearning process. To assess the effectiveness of unlearning methods, we use three main types of verification: attack-based, accuracy-based, and information-based, as categorized by [4]. Information-based verification measures the difference between the original and unlearned models using metrics like KL divergence between output distributions [5] and L2 distance [3] between model parameters.

Additionally, the effectiveness of unlearning can be evaluated by its resilience to adversarial attacks. We explore 2 common types of privacy attack in our project: membership inference attacks (MIA) [6] and data reconstruction attacks [7]. MIAs attempt to identify whether particular data points were included in the model’s forget set. This creates privacy risks, as successful MIAs can expose which specific information was meant to be forgotten, revealing sensitive user data. MIAs require attackers to have candidate data points to test against the model, while reconstruction attacks pose a greater threat, aiming to directly regenerate the forgotten data, a more severe privacy breach.

These attacks are classified as black-box or white-box based on the attacker’s knowledge. Black-box attacks assume no access to model parameters, architecture, or training data, while white-box attacks assume full access. Many white-box attacks exploit model gradients, which capture how parameters change during training or unlearning [8].

1.4 Motivation

Machine unlearning currently lacks standardized frameworks [9]. Existing repositories offer limited customizability, supporting specific models and datasets and predefined hyperparameter configurations, as repositories are mostly designed to reproduce experimental results [10][11][12]. Recent research highlighting privacy risks in approximate unlearning [13] suggests a useful machine unlearning framework must also offer evaluation against a variety of attacks.

Our application, SafeUnlearning, addresses these gaps with a configurable end-to-end pipeline for image-based machine unlearning. It covers dataset processing, model initialization, pretraining, unlearning, score evaluation and privacy attack evaluation. SafeUnlearning lets users specify which workflow components they need through simple terminal commands, making the unlearning process adaptable and easy to run. Moreover, it allows the user to easily extend the repository by customizing any part of our end-to-end pipeline.

1.5 Existing work

Since machine unlearning is still an emerging research field, most existing repositories prioritize validating novel methods as opposed to providing standardized comparison frameworks or generalizable code. While they may offer some flexibility, they typically require substantial additional engineering before systematic evaluation. Many projects are set to run an entire preconfigured pipeline from start to finish [12], however, our modular software architecture allows users to break down the machine unlearning workflow into several steps, preserving checkpoints like dataset splits for reuse across different algorithms which is particularly valuable for research requiring iterative hyperparameter adjustments.

The repository “Towards Unbounded Machine Unlearning” [14] provided a foundation for our work, which implements most of the unlearning methods in our repository, except our novel method (SGRU 4), as well as BAD-T, from which SCRUB was inspired. We extend this by supporting many Pytorch image classification architectures for standard datasets, enabling users to use custom models or download pre-trained torchvision, timm and Torch Hub models. Similarly, Cadet’s repository [10] provided valuable structural insights with its comprehensive evaluation framework, including the state-of-the-art LiRA attack and multiple unlearning methods.

2 Logistics

2.1 Team management

The 3 primary goals of our project were: 1. building a machine unlearning repository and propose a novel unlearning algorithm based on model parameters, 2. adapting SOTA MIAs to unlearning, 3. Adapting reconstruction attacks from federated learning to unlearning. Andrew and Bobbi took charge of repository design and core app features, with Bobbi focusing on conducting research into a novel unlearning algorithm and integrating it into the repository. Jebastin and Lynn focused on MIA, while Ola and Varsha worked on reconstruction attacks. Each task required both in-depth research and strong software skills, as we had to explore and modify existing repositories, as well as write new code to fit our research needs.

During term time, each member contributed approximately 10 hours per week, focusing primarily on research and running experiments. The spring break marked a shift to implementation work, including initializing the unlearning repository, integrating existing unlearning methods, and building support for training, unlearning, custom dataset loading, and running configurable experiments. As the repository matured, all experiments ran through the application, with group members reporting bugs, making feature requests, and contributing to its development. Towards the end of the project, members worked to integrate their research into the repository, resulting in a cohesive final product.

To ensure effective collaboration, the team held two weekly meetings: one with the supervisors to share progress updates and receive new directions, and another internal meeting between group members to discuss findings, resolve doubts, and support each other. During the break, we transitioned to working full-time and held daily stand-ups to get updates from each team member and keep the project moving at a steady pace to meet all deliverables. A Microsoft Teams group that included the supervisors was created for project discussions, research sharing, asynchronous updates, and meetings scheduling. We used Trello to assign tickets to group members and track their progress over each sprint. Separate boards for research and software development were created to organize tracking.

2.2 Software management

The provided GitLab group was used as the primary hub for managing all of the code and developing the main repository. Following industry standards, we created *master*, *staging* and *dev* branches. We additionally opted for a feature branching strategy to isolate feature-related bugs and improve code stability. Feature development took place on *dev*, while the *staging* branch was for testing more stable code. Merge requests were created to integrate new features or bug fixes into *dev*, with at least one team member reviewing the changes.

PEP8 standards were followed throughout the development process, with the help of Flake8, *ruff* python linter, and *isort* for import sorting. Unit tests were written for the core functions and added to GitLab continuous integration (CI) for testing to ensure that new merges would not break core functionality. To ensure code readability, some standard software engineering practises such as Google style docstrings, and type-hints and comments wherever helpful were followed.

3 Technical Decisions, Design, and Architecture

3.1 SafeUnlearning overview

SafeUnlearning¹ is a Python framework for running image-based machine unlearning jobs, designed with user customizability in mind. We identified five key steps of the machine unlearning process and created five distinct mini-applications for each of these steps:

1. Dataset splitting
2. Model pretraining
3. Unlearning
4. Score evaluation
5. Privacy risk evaluation

While steps 1–5 are generally required for comprehensive machine unlearning experimentation and evaluation, users may specify their needs to minimize computation.

Our repository is broadly structured as follows:

```
safe-unlearning/  
|- configs/  
|- demos/  
|- scripts/  
|- src/  
|  |- attacks/  
|  |- datasets/  
|  |- evaluation/  
|  |- train/  
|  |- unlearning/  
|- tests/
```

We adopted a `src/` layout for our repository to better distinguish application source code and experiment code. The `demos/` folder contains experiment runs using SafeUnlearning, performing our adapted privacy attacks on benchmark datasets and models. The `src/` folder contains our source code, and each subfolder corresponds to one of the 5 mini apps that can be run from a `main.py` within each subfolder. Shell scripts were added to the `scripts/` folder to test run the algorithms present in the repository.

¹<https://gitlab.doc.ic.ac.uk/g24mai07/safe-unlearning>

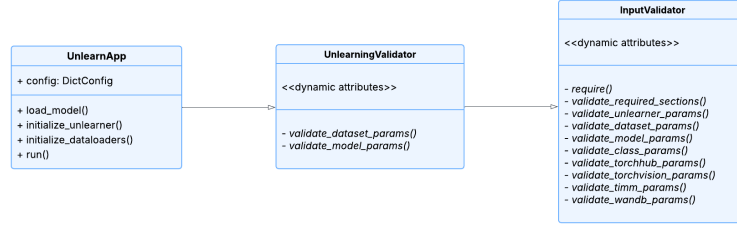


Figure 1: 2 layers of configuration validation for the unlearning app. The UnlearningValidator methods are specific to the unlearning app, while the InputValidator methods can be used by other mini-apps.

Our system requires user input via command line arguments to run all mini-apps. Under the hood, the Hydra framework parses these command line arguments into corresponding configuration keys before producing a configuration dictionary input to each mini-app. Hydra enables users to specify mandatory keys or override defaults directly from the command line, minimizing additional engineering work. To minimize complicated downstream errors related to the configuration, we implemented a custom exception `ConfigError` and used `MissingMandatoryValue` from the OmegaConf library, as well as look-before-you-leap (LBYL) configuration validation. This LBYL-style configuration validation was implemented using two levels of class inheritance: a global configuration validator at the base level and an app-specific configuration validator that extends it for application-specific checks. A UML diagram of this class inheritance structure for the unlearning app is displayed in Fig. 1.

A global configuration validation class was introduced because multiple mini-apps, such as the training and unlearning apps, share common validation requirements for key inputs like model loading. However, app-specific configuration validation is also necessary to handle cases where different apps require users to specify different details for the same key inputs. For example, the datasets app requires users to specify a forget set creation method within the dataset configuration, whereas the unlearning app only requires a dataset name and batch sizes for dataset loading and transformation. For computational efficiency, the configuration check process is also intertwined with attribute assignment. For instance, the global-level `_require()` method raises a `ConfigError` if the configurations dictionary is missing a necessary key. If the key is present, however, the `_require()` method instantiates a corresponding class attribute with the key’s value.

Each mini-app prints the final Hydra configuration to the terminal, allowing users to review the configuration that is being executed.

Our system also features optional W&B integration that logs useful details such as the configuration, performance across epochs and time taken for model inference over different splits, helping the user to track experiment runs and identify computational bottlenecks.

3.2 Dataset app

Our dataset app is run from the `datasets/main.py` file. Datasets must be obtained either from a local directory or downloaded as CIFAR datasets through torchvision. Local datasets require ImageFolder format with folder names representing label indices. When downloading through torchvision, the system initializes datasets in the specified directory using ImageFolder format, assigning unique filenames to each image. The system creates an output folder that contains the train, validation, forget, and retain splits, along with a test split if the initial dataset includes a `tests/` folder. Validation, forget, and retain splits are derived from the training data. To minimize storage requirements for benchmark datasets, the output folder contains only symlinks to the original data rather than duplicates. The splitting process and symlink implementation are illustrated in Fig. 2, while the overall application flow appears in Fig. 3.

To minimize unexpected downstream bugs from experiment run variability, the dataset app clears the output directory before generating new random splits. Although this requires recomputing splits

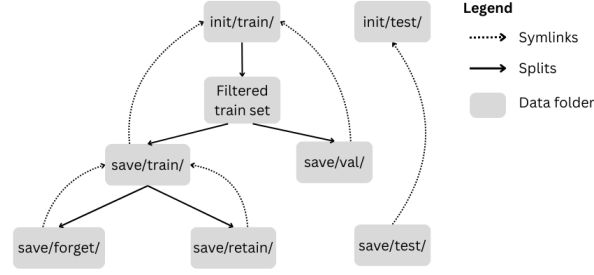


Figure 2: Dataset app splitting process. Dotted lines indicate symlinks from a data folder to source.

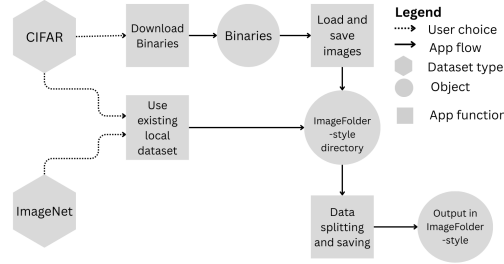


Figure 3: Dataset app flow from dataset type to output. The user can only download CIFAR datasets.

if rerun, users can avoid unnecessary recomputation by saving the generated splits once and reusing them in subsequent steps.

The dataset app accommodates diverse machine unlearning requirements, supporting multiple forgetting strategies: entire classes, random instances, a specified number of random instances from certain class(es), or particular data points. The app allows defining a subset of remaining training data as the retain set, reducing computational requirements and simulating possible scenarios. Additional configuration options include specifying the proportion of initial training data to use, the validation set ratio, and the output splits directory path.

Currently only subsets (including the full subset) of CIFAR5, CIFAR10, CIFAR100, and ImageNet are supported. While dataset splitting is inherently dataset-agnostic, this limitation exists because subsequent mini-applications require predefined transformations to process raw images into input tensors. As we believed it would be confusing to allow varying degrees of customisability in the entire application especially for users who want to run all 5 mini apps, we limited the number of supported datasets. Extending the SafeUnlearning to support other datasets should be straightforward and requires only adding an additional dataset transformation file and minimal code changes. Extending SafeUnlearning to handle more custom datasets could potentially involve expanding the configuration to allow the user to pass in their dataset’s transformation parameters.

3.3 Train app

Our train app is run from the `train/main.py` file. It allows users to initialize and pretrain a model on training data while iteratively saving the best model’s weights based on validation loss to an output file at each epoch.

Model architectures may be loaded in for training from torchvision, Pytorch Image Models (TIMM), Torch Hub, and the local drive (given the user also has the model architecture for initialization). This is implemented through an `ImportModel` class containing the necessary methods. For models downloaded through torchvision, the default classification head for ImageNet is changed to a linear layer with output size equal to the number of classes if the user is not training on ImageNet.

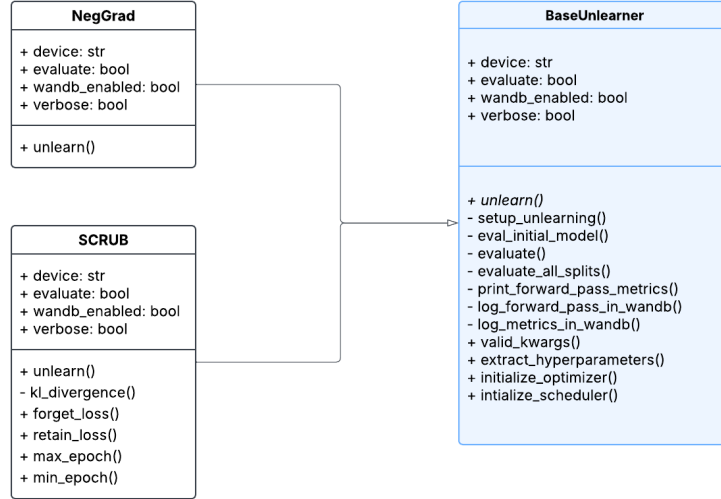


Figure 4: Example class relationship between unlearning algorithms SCRUB and NegGrad to the superclass, BaseUnlearner. Training setup and logging methods have been refactored into the superclass, leaving only algorithm-specific methods in the subclasses.

This configurability covers a wide range of unlearning contexts, from training a custom model architecture to downloading an image model pretrained on ImageNet from torchvision and performing transfer learning on CIFAR10. Typical training configurations are customizable, such as learning rate, optimizer, weight decay, and number of epochs.

As training can be a computationally expensive and iterative process, this app is also designed to be able to resume training and logging from a previous run, just by the user specifying the same W&B run ID and using the weights checkpoint saved in the previous run.

3.4 Unlearning app

The unlearning app is run from the `unlearning/main.py` file. It initializes dataloaders from dataset splits in the local drive, as well as a user-specified model. Similarly to the train app, the same `ImportModel` class is used for model loading, thus allowing the unlearner app to initialize the same set of models as the train app. Therefore, the user can skip the train app if they wish to perform unlearning immediately on downloaded pretrained weights. The unlearning app can also load locally saved weights into model architectures, which works directly with the output of the train app. In addition to model loading and dataset parameters, the user must also specify their desired unlearning algorithm, and can override any defaults. This app then performs unlearning on the initial model and saves the unlearned model weights and training logs to the local drive.

The structure of our unlearning app takes inspiration from Cadet’s repository [10], which contains a range of unlearning algorithms implemented in an object-oriented style, with a `BaseUnlearner` superclass. However, we built each unlearning algorithm present in our repository directly from the source research papers, taking reference from [10] where necessary regarding implementation challenges, such as handling mismatches in the number of batches in the retain and forget sets by cycling the forget set.

We extended `BaseUnlearner` by forming a foundational group of superclass methods that was shared by every unlearning algorithm. This included evaluation, optimizer and learning rate scheduler initialization, and results logging. To illustrate the structure of our unlearning methods, Fig. 4 presents a subset comprising a simple method (NegGrad), a more complex method (SCRUB), and their common base class (BaseUnlearner), reflecting the refactoring principles we adopted.

Each unlearning algorithm requires a different set of hyperparameters, which added complexity. For

instance, NegGrad+ has a β parameter that controls the tradeoff between retain and forget loss. SCRUB uses α and γ to weight different kinds of retain set loss, as well as requiring specification of the number of forget (max) / retain (min) steps. The Hydra configuration framework helped manage this complexity by allowing us to create separate YAML configuration files for each algorithm. This approach minimizes redundant arguments, eliminates potentially misleading unused keys, and makes the experiment configuration more intuitive for the user.

We implement finetuning, EU-k, CF-k, NegGrad, NegGrad+, SCRUB, and SGRU, using a consistent class structure across methods to improve clarity and aid in understanding the underlying unlearning processes. This structure also helps identify the key differences between the methods. Each of these methods requires one or both of the forget and retain datasets for unlearning. The general `unlearn` method performs a training loop, where the model is optimized according to a loss function. The loss function guides the unlearning process. To enhance code clarity, we use helper functions where needed, such as defining the loss functions for NegGrad+ and SCRUB outside the unlearning loop, and abstracting SCRUB’s min/max steps.

A technical decision we made was to compute all splits’ evaluation losses and accuracy metrics at the end of an epoch, as opposed to more efficiently storing and averaging losses over the epoch. While this is more computationally expensive, experiments on ResNet18, ResNet20 and MobileNetV2 models over realistic CIFAR10 retain/forget splits indicated that the computational bottleneck was in the unlearning forward pass, rather than the evaluation pass. Therefore, in the interest of fair comparisons, we opted to recompute all losses at the end of an epoch. To cater to users with lower computational resources, we added a key in the unlearner configuration to allow them to skip the evaluation step at the end of each epoch. One area for future work is to avoid shuffling the retain and forget sets during evaluation. This is necessary during the unlearning forward pass, but unnecessary during evaluation. If unnecessarily shuffling data can be avoided, the evaluation step time cost would be further reduced.

3.5 Evaluation app

This app evaluates unlearned models using accuracy and information-based metrics. It handles one original model and multiple derived unlearned models, with user-selectable data splits for evaluation. The app generates a result file, accuracy bar plots for all models, and comparative visualizations showing differences between unlearned models and the original using metrics like L2 and KLD, facilitating effective model comparison.

3.6 Attacks app

The attacks app provides a framework for executing both MIA and reconstruction attacks through the command-line interface. The user can specify the name of the attack, as well as parameters in the configuration, making the structure flexible and easy to adapt. The app follows a modular architecture, enabling independent development and execution of distinct attack methods while maintaining consistency with the existing repository architecture.

Reconstruction attacks are called through the shared `ReconstructorApp` class by executing `attacks/main_reconstructor.py`. This class inherits from `ReconstructorValidator`, which alongside `GLIRValidator`, is within created `config_validation.py` to ensure the valid input configuration for the reconstruction attack.

`ReconstructorApp` is responsible for handling the loading of original and unlearned models from designated checkpoints, initializing Weights & Biases (wandb) for experiment tracking, and instantiating the reconstructor before executing the reconstruction process via the `reconstruct` method. Moreover, we introduce methods for evaluation, including execution time measurement, calculation of PSNR and MSE metrics from `metrics.py`, and storage of generated images together with their performance metrics.

4 Developing a New Machine Unlearning Method

4.1 Description

Researchers have proposed various approximate machine unlearning methods. A naive example is fine-tuning on retain data. However, this often leads to overfitting without guaranteeing effective forgetting. Variants of this approach include Catastrophic Forgetting-k (CF-k) and Exact Unlearning-k (EU-k) [15], where the first k layers of the model are frozen (with others reinitialized in EU-k) before fine-tuning. NegGrad uses gradient ascent on the forget set to degrade the model’s performance on the forget set. However, this can also degrade model utility. NegGrad+ addresses this by combining gradient ascent on forget data with gradient descent on retain data, balancing forgetting and knowledge preservation. The current SOTA method, SCRUB, implements an oscillating optimization strategy: a maximization step applies gradient ascent on KL divergence of model outputs on the forget set, while the minimization step uses gradient descent on both KL divergence and classification loss to maintain performance while ensuring effective forgetting.

4.2 Subspace Gradient Redirection Unlearning

We developed Subspace Gradient Redirection Unlearning (SGRU) to address vulnerabilities in existing methods which often creates distinctive optimization patterns that remain detectable to adversarial attacks. Rather than directly optimizing against specific examples, SGRU systematically removes gradient components associated with forgotten data using Singular Value Decomposition (SVD).

SGRU first identifies principal gradient directions in the forget set’s parameter space. For each layer l , we compute and flatten gradients g_i^l from forget samples to form the forget gradient matrix:

Specifically, for each layer l , we compute gradients g_i^l for a batch of forget samples (x^f, y^f) and flatten them to obtain g_{fi}^l , where $i \in \{1, \dots, b\}$ and b is the number of forget batches. These are concatenated to form the forget gradient matrix for which we apply SVD. The matrix V^l contains the eigenvectors of the forget gradient space. We choose the top- k directions to form:

$$G_f^l = [g_{f1}^l, g_{f2}^l, \dots, g_{fb}^l] = U^l S^l (V^l)^\top \in \mathbb{R}^{d \times b}$$

$$M_f^l = [V_{f1}^l, V_{f2}^l, \dots, V_{fk}^l] \in \mathbb{R}^{d \times k}$$

M_f^l is stored for each layer. Gradients from the retain set are then computed and redirected by projecting them orthogonally to this subspace. For each batch from the retain set D_r , the new redirected gradient is given by:

$$\nabla \mathcal{L}_r^* = \gamma \nabla \mathcal{L}_r^l - \lambda \nabla \mathcal{L}_r^l M_f^l (M_f^l)^\top$$

Where γ is the weighting of the retain gradients and λ is the redirection strength parameter that controls the extent of projection.

SGRU offers precise control through several hyperparameters. The `recalc_freq` parameter determines how often the forget subspace is recomputed. Recalculating too frequently increases computation cost, while too infrequently may lead to stale approximations. The `num_components` parameter specifies how many principal components are extracted from the forget gradients, bounded by the number of forget batches and gradient dimensions. The `redirection_strength` λ scales gradient redirection intensity—0 equals fine-tuning, 1 applies complete orthogonal projection. Higher values push gradients further from forget directions. The `retain_strength`, γ , weights the retain gradients. This is to allow further control on the utility/forget quality tradeoff, or allow for unlearning without the retain set.

5 Adapting Reconstruction Attacks from Federated Learning

5.1 Description

We adapt two attacks from federated learning (FL): Generative Gradient Leakage (GGL) [16] and inverting gradients [17]. FL allows multiple clients to collaboratively train models without directly sharing data by sending gradient updates to a central server. Attackers can exploit these gradients to reconstruct private training data. However, in white-box unlearning, we no longer have access to explicit gradient updates, only the original and unlearned model weights. This requires reformulating the attack to work with model weight differences rather than direct gradients. In both FL and unlearning, the attacker has access to the model architecture, some form of differential information, and can pose reconstruction as an optimization problem.

5.2 Implementing reconstruction attacks

The attacks follow the same structure as other applications in the repository. The code is validated by replicating original reconstruction experiments to establish a baseline before introducing unlearning-specific constraints.

5.2.1 Evaluation metrics

Beyond visual comparison, we quantitatively evaluate reconstructions using Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR) [16][13]. MSE measures pixel-wise differences, while PSNR captures the ratio between maximum pixel intensity and MSE. For batches larger than one, we pair each ground-truth forgotten image with all reconstructions, reporting the lowest MSE and highest PSNR. Following [13], we report the best-performing reconstructions, as even a single accurate recovery can threaten privacy.

5.2.2 Attack assumptions

The attacker is assumed to have access to the model architecture, parameters, and the number and labels of the unlearned samples. We also assume that the attacker knows the data mean and standard deviation, which is reasonable as these can typically be estimated from an auxiliary dataset.

5.3 Generative Gradient Leakage

In federated learning, GGL [16] reconstructs private training data from shared gradients by leveraging pretrained generative models. GGL conditions generative models like BigGAN on inferred labels and uses gradient-free optimization to generate images that match observed gradients. We adapt GGL to the machine unlearning context with a Turbo optimization process. Our implementation generates candidate images via BigGAN, simulating unlearning on the original model using these images, and uses Bayesian optimization to minimize differences between the simulated and actual unlearned models.

5.3.1 Implementation details

To adapt GGL to machine unlearning, several key modifications were required:

- **Choice of the optimisation objective:** In unlearning, attackers may not have access to explicit gradients. Therefore we redefine the optimization objective to minimize the parameter difference between the unlearned model θ_u and a simulated model θ_n . Starting from the original model θ_o , we simulate the unlearning update using the candidate image and compare the resulting θ_n to θ_u . This required introducing `reconstruct` and `evaluate_loss` methods.
- **Loss Functions:** To evaluate θ_u vs. θ_n , we implement a set of user-selectable loss functions, including: L1 norm, L2 norm, a weighted variant emphasizing later layers, and an interpolated loss across different local step counts.

- **Model updates:** To mimic unlearning, we introduced `perform_sgd_updates`, `perform_neggradplus_updates` and `perform_scrub_updates`, which apply NegGrad and SCRUB unlearning to the model using the generated image.
- **Checkpointing:** To monitor and resume optimization reliably, we save latent vectors in an `optimize` function inside `turbo_1.py` every time a better image is proposed by the optimiser. We extend the `GGLReconstructor` class with a `save_results` function. This records the image and latent vector returned by reconstructor, and also regenerates all images from saved latent vectors during optimisation in order to visualize progress.
- **Gaussian process customization:** Through experimentation, we observed that optimizer choice, learning rate, and scheduler of Turbo optimization significantly affect reconstruction quality. We made these parameters configurable from a YAML configuration file, customising `train_gp` function inside `gp.py`.

5.3.2 Results

Our adapted GGL method on ResNet-18 and ImageNet data successfully reconstructs key features of the original image, such as object shape, position, and dominant colors. As shown in Figure 5, early reconstructions capture the general structure, while longer optimization leads to more background detail. Fine-grained textures, like the rocks or food, remain challenging.

Despite resource constraints limiting optimization steps, results indicate meaningful leakage. We evaluate reconstruction quality across increasingly complex unlearning scenarios, with consistent target image reconstruction for comparison. The reported metrics show comparable or better performance than those in [16], highlighting the effectiveness of our implementation. The results shown are after implementing Turbo for a few thousand steps, with metric scores and pictures obtained from wandb logging system.

Unlearning type	PSNR $\uparrow$	MSE $\downarrow$
Neggrad, 1 update	8.70	0.130
Neggrad, 2 updates	8.43	0.143
Neggrad, 3 updates	7.97	0.159
Scrub (no retain), 1 max_steps	7.50	0.178
Scrub (no retain), 2 max_steps	7.11	0.19
Scrub (no retain), 3 max_steps	8.00	0.16

Table 1: Reconstruction performance across unlearning types

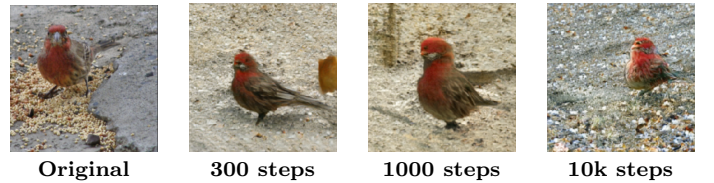


Figure 5: Reconstruction progression for 1 Neggrad update

5.4 Inverting gradients

Inverting gradients is based on the federated learning attack “Inverting Gradients – How easy is it to break privacy in federated learning?” [17], which was later adapted for machine unlearning by Hu et al. [13]. However, this set-up only considers single-step gradient ascent without any retain data. To evaluate more complex unlearning methods, we extend the attack to support multiple optimization steps and introduce a retain set. The attack reconstructs a dummy image, x' , by optimizing it so that gradients, when passed through the original model, closely match the gradients of the true unlearned samples, ∇' . When successful, the resulting x' resembles the forgotten data.

This method is based on gradient inversion approaches such as “Deep Leakage from Gradients” [7] but extends it with a cosine regularization term, since gradient directions in high dimensions reflect semantic differences. This essentially adapts the attack to reconstructs input that result in similar predictive changes as the groundtruth sample, rather than exactly match gradients. Total variation loss (TV) is also included to promote spatial smoothness in the recovered images.

In the unlearning setting, we estimate the gradient of the loss of unlearned samples, ∇^* , using the the differences in the original and unlearned model parameters [13]. As such, the function we aim to

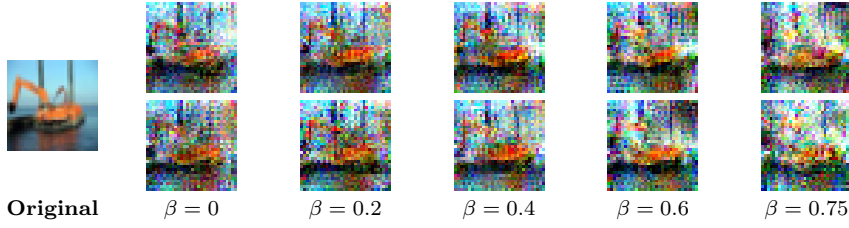


Figure 6: Image reconstruction for varying β values across 1 NegGrad+ epoch with a forget batch of 16 and a retain data size of 50. Reconstructions from two experimental runs are shown for each value.

optimize is:

$$\mathcal{L}_{\text{recon}} = -\cos(\nabla', \nabla^*) + \alpha \cdot \text{TV}(x')$$

where,

$$\nabla' = \frac{\partial \mathcal{L}(f_{\theta}(x'), y')}{\partial \theta}, \quad \nabla^* = \theta - \theta_u$$

and x' , y' the dummy image and label, while x , y are the ground-truth samples.

5.4.1 Implementation details

The attack uses the original model, before unlearning, because it contains the gradients associated with the image. The `InvertGradReconstructor` class takes both the original and unlearned models to compute the weight difference, which is used in the cost function. Dummy input images are initialized with noise or zeros and normalized before being passed to the original model. The normalization is a key alteration from the original attack that significantly improves reconstruction quality.

We also consider an alternative loss combining the KL divergence between the current model’s predictions and a frozen copy of the original model passed in, with the cross-entropy loss. This was designed to mimic and effectively attack SCRUB, but has also been successful on other unlearning methods.

5.4.2 Results

Beta	0	0.2	0.4	0.6	0.75
Min MSE	0.038	0.046	0.040	0.043	0.063
Max PSNR	14.21	13.39	14.00	13.69	12.01

Table 2: Minimum MSE and maximum PSNR values for inverting gradients attack on Neggrad+ with batch size 16, 1 unlearning epoch, and 50 retain, varying β . Here, β controls the weighting of the retain loss in the final objective, while $1 - \beta$ weights the forget loss.

Experiments are run on a ResNet18 trained on CIFAR10, varying unlearning epochs, batch sizes, and unlearning-specific hyperparameters. Attack settings follow previous work [13][17].

NegGrad: The attack remains effective across different unlearning epochs (1–3) and batch sizes (1, 16 and 32). Larger batches lead to noisier reconstructions and reduced PSNR, but some samples reconstruct well.

NegGrad+: Introducing a retain set with a β hyperparameter degrades reconstructions as β increases (Table 2). By $\beta = 0.6$, fine details vanish, and by $\beta = 0.75$, outputs are mostly noise, although some features like shape or color still leak. Different samples leak different features, allowing image details to be combined across runs (Fig. 6).

SCRUB: Max steps use KL divergence on the forget data, producing weak, noisy gradients and poor reconstructions. Changing SCRUB’s forget loss to KL + CE significantly improves results, particularly for small batches. Larger batches reconstruct only a few samples well, suggesting SCRUB’s strength partly lies in its loss design.

SGRU: On all runs, without gradient deflection on the retain set, the forget loss is unchanged and the method fails to forget. When deflection is enabled, the attack recovers only the retain images, showing that the forgetting method relies on effective gradient deflection.

6 Adapting Membership Inference Attacks

6.1 Likelihood Ratio Attack

Likelihood Ratio Attack (LiRA) was introduced in [18], which views membership inference as a hypothesis test - whether a model θ was trained on a target point (x, y) or not. Let $\theta \leftarrow \mathcal{T}(\mathcal{D})$ represent the model produced by applying a training method $\mathcal{T}$ to a dataset $\mathcal{D}$. Consider $\mathbb{Q}_{in}(x, y) = \{\theta \leftarrow \mathcal{T}(\mathcal{D} \cup (x, y)) \mid \mathcal{D} \subset \mathbb{D}\}$ to be the distribution of models trained on datasets containing (x, y) , and $\mathbb{Q}_{out}(x, y) = \{\theta \leftarrow \mathcal{T}(\mathcal{D} \setminus (x, y)) \mid \mathcal{D} \subset \mathbb{D}\}$ be the distribution of models trained on datasets not containing (x, y) . Using the Neyman-Pearson lemma, the Likelihood-ratio Test between the two hypotheses is given as:

$$\Lambda(\theta; x, y) = \frac{p(\theta(x)[y] \mid \theta \in \mathbb{Q}_{in}(x, y))}{p(\theta(x)[y] \mid \theta \in \mathbb{Q}_{out}(x, y))}$$

The distributions $\mathbb{Q}_{in/out}$ are approximated as univariate gaussians, and several “shadow models” are trained to estimate the four parameters: the mean and variance of each distribution. Finally, to perform the attack on any model θ , the likelihood of the model output $p(\theta(x)[y])$ is computed using both the distributions, and is assigned to either the “in” or “out” distribution based on whichever likelihood is larger.

6.1.1 Adapting LiRA for unlearning

LiRA adapted for unlearning setting was proposed by [14], called as U-LiRA. For each shadow model, a corresponding “shadow unlearned” model is generated, by applying some unlearning method on the original model. $\mathbb{Q}_{in}$ now represents the distribution of unlearned models where the sample (x, y) was forgotten. The attacker predicts that the sample (x, y) was forgotten if its likelihood under $\mathbb{Q}_{in}$ is larger than $\mathbb{Q}_{out}$.

6.1.2 Implementing U-LiRA

The attack’s implementation follows a similar structure to the other apps in our repository. **LiRAApp** serves as the main orchestrator for the entire pipeline, handling data splitting, training original and unlearned shadow models, and computing the membership score. It inherits from **LiRAValidator**, which acts as the validation layer for the attack, and initializes all the necessary class attributes. The main steps for the attack are implemented as separate functions, and called by the **LiRAApp**:

1. **Data splitting** - Creates the three types of splits: train-test splits for shadow models, retain-forget splits within training data and validation split for model evaluation. The indices are stored as numpy arrays for use in further steps.
2. **Training shadow models** - The **UnlearnAppForLiRA**, inherited from **UnlearnApp**, handles all the training and unlearning of shadow models. We initialize the class once, and call its **run()** for every split-forget pair. Once the models are trained, we compute, and store the output probabilities for all splits and their corresponding shadow models.
3. **Computing the score** - The score is computed holistically, considering all forget points. At each step, one split-forget pair is treated as the original models, while the remaining N-1 pairs serve as shadow models. For each point in the forget set, we collect the output probabilities from the shadow models, both when the point appears in the forget set and when it appears in the test set — and store these separately in **ProbasNeverAndForgotten**. We then compute the mean and standard deviation of these lists to estimate the distributions $\mathbb{Q}_{in}$ and $\mathbb{Q}_{out}$. Using these distributions, we calculate the likelihood of each point belonging to them using the original model outputs, and store the resulting ratio, which becomes the membership score for that point using one split-forget pair. This process is repeated across all N split-forget pairs, and the final membership score is obtained by averaging the individual scores.

6.1.3 Extending U-LiRA

The attack, as proposed in [14] only considers the outputs from the shadow unlearned models. We aim to extend this attack by also utilizing the outputs from the shadow original models. A straightforward approach is to model Q_{in} as a bivariate Gaussian distribution between the shadow original and unlearned models, which helps to capture their relationship. This extension is quite simple to implement, as the original shadow models and their predictions are already generated, with the only difference being estimating Q_{in} as a multivariate normal with parameters μ and Σ .

6.1.4 Results

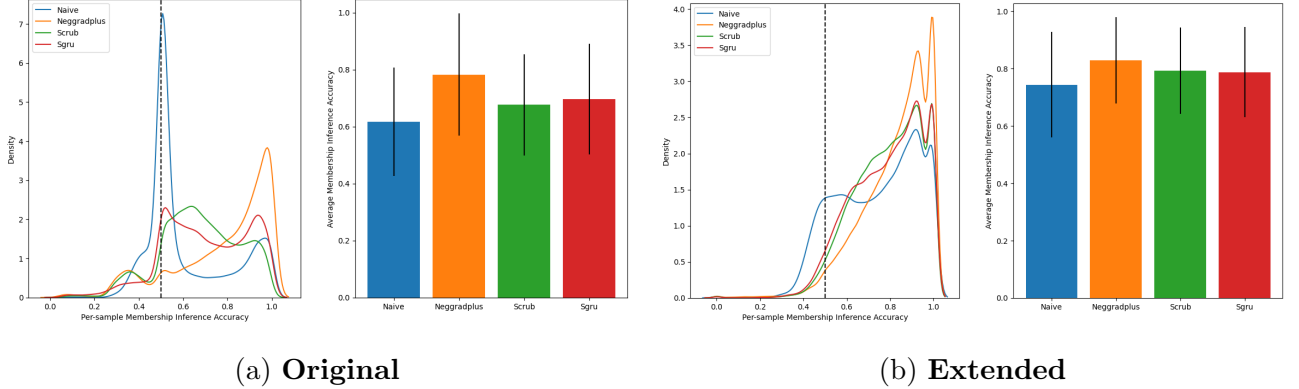


Figure 7: KDE and Bar plots for U-LiRA against different unlearning methods.

We used U-LiRA implemented in our repo to evaluate 3 unlearners (NegGrad+, SCRUB, SGRU), and compared them with the exact unlearning baseline (Naive). Following research trends, we used a pretrained ResNet18 and CIFAR10 dataset to run our experiments. From Fig. 7.a, we observe that naive unlearning exhibits a sharp peak around 0.5, which is the best case scenario, as predicting the membership of a target sample is equal to random guessing. NegGrad+ is the weakest unlearner among the 3, based on its high membership inference accuracy, while SGRU shows comparable performance to SCRUB with minimal hyperparameter tuning, showcasing its promise as a strong unlearning method.

From Fig. 7.b, our extended U-LiRA increases the membership accuracy across all unlearners, indicating a significant privacy leakage. SGRU shows the best defence against this attack, compared to other approximate unlearning methods. While adding information about the original shadow models helps improve the attack success, it comes with the added assumption of the attacker having access to both the original and unlearned model, which may not always be true. Relaxing this assumption, while maintaining the strengths of this extended attack is an ongoing research direction.

6.2 Gradient Likelihood Ratio (GLiR) attack

The Gradient Likelihood Ratio Attack (GLiR) [19] is a white-box approach requiring access to model parameters at each training step. It analyzes gradient distributions to distinguish between training and non-training data. GLiR estimates the mean μ and inverse covariance matrix Σ^{-1} of gradients using background data, then computes the gradients of query points with respect to model parameters θ' . The attack calculates test statistic $\hat{S}$ and gradient susceptibility $\hat{K}$:

$$\hat{S} = (n - 1)(m_t - \theta')^T \hat{\Sigma}^{-1} (m_t - \theta')$$

$$\hat{K} = \left\| \hat{\Sigma}^{-1/2} (\theta' - \hat{\mu}) \right\|_2^2$$

Which denotes the squared Mahalanobis distance between query and background gradient distributions. GLiR aggregates p-values across training steps as attack scores, comparing them with threshold η .

6.2.1 Adapting GLiR for unlearning

In this project we adapt the white-box attack, GLiR, for unlearning. Three key modifications have been made to the attack. First, instead of collecting gradients over multiple training/unlearning steps, only the original and final unlearned model parameters are used for calculations, simulating a more realistic setting. Second, GLiR uses the gradient differences between the models, $\Delta\theta'$, to capture how much each query point's influence on the model has changed as a result of unlearning. Third, we add a novel component that considers the ratio of normalised $\Delta\theta'$ and $\theta'_{\text{original}}$ to magnify the separation in the test statistic distributions between the two sets of data points. We define two statistics:

$$\text{Diff Score} = \Delta\theta' = \theta'_{\text{original}} - \theta'_{\text{unlearned}}$$

$$\text{Ratio Score} = \left[\frac{\|\Delta\theta'\|}{\|\nabla_{\theta}\mathcal{L}(f_{\text{before}}(x), y)\| + \varepsilon}, \frac{\|\nabla_{\theta}\mathcal{L}(f_{\text{before}}(x), y)\|}{\|\Delta\theta'\| + \varepsilon} \right]$$

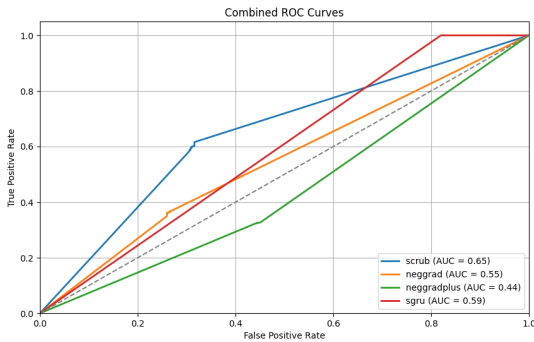
We computed baseline distributions from test statistics using a held-out test set. For evaluation, we created balanced query sets of forgotten and test points as a binary classification problem, measuring attack performance via accuracy, precision, recall, and AUC. Strong attack classification suggests the model retains information about forgotten data despite unlearning. Optimal unlearning balances good performance on retain/test data, reduced performance on forget set, and robustness against MIAs.

6.2.2 Implementing GLiR

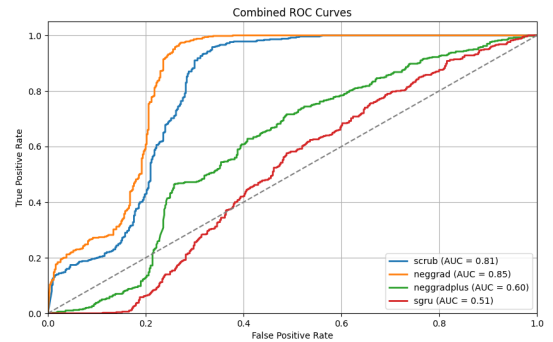
GLiRApp performs all elements of the attack, and uses test data to create a baseline for the statistics. It inherits from the GLiRValidator class, which validates the attack configurations. Its core functionalities are:

1. **Data preparation** — Create a baseline using same-distribution data. Form query points from the forget and test sets, either balanced (if many forget points are available) or with a larger test proportion (if the forget set is small). If the forget set is large, the user can leave the `test_size` config value empty to create a balanced set. If the forget set is small, the user can instead select a larger test set and focus on precision and recall rather than overall accuracy.
2. **Creating the Attack** — Load both the original and unlearned models and calculate the test statistics for the background data using gradients for each point, storing the mean and variance.
3. **Membership Evaluation** — Evaluate the p-values returned from the likelihood ratio test. The user can choose their threshold for classification.

6.2.3 GLiR Results



(a) **Difference-based**



(b) **Ratio-based**

Figure 8: ROC curves for GLiR attacks against different unlearning methods. These figures were directly exported from UnlearnApp without additional modification.

Experiments were conducted using a pretrained ResNet20 on CIFAR10 on the Unlearn app. GLiR has been evaluated using 4 unlearning methods, SCRUB, NegGrad, NegGrad+ and SGRU. As shown in Fig 8, the GLiR attack is effective across all unlearning methods tested. Overall, the Ratio-based method results in a higher AUC than the Difference-based method, and provides better discrimination between forgotten and test points. Compared to the other three unlearning methods, NegGrad+ and SGRU appears more robust against GLiR, as its ROC curves lie closer to the diagonal line, which represents random guessing.

7 Conclusion

With SafeUnlearning, we have developed a highly configurable machine unlearning and privacy evaluation framework, which will help advance research in machine unlearning and its evaluation. We have used this system to support a wide range of unlearning experiments, accelerating our research.

7.1 Research milestones

We successfully adapted attacks from federated learning and membership inference to the machine unlearning problem. The effectiveness of the 4 attacks highlights vulnerabilities in current unlearning techniques and motivates the need for more robust, privacy-preserving unlearning methods. Our work highlights how we took established methods and extended their relevance to unlearning in privacy research.

The extended U-LiRA attack reveals that with a simple modification, having access to the original model substantially increases privacy risks from existing unlearning methods. GLiR proves particularly effective at identifying membership status in certain unlearning methods when white-box access is available. The addition of the two reconstruction attacks provides complementary information. GGL shows meaningful results in generating realistic images using GAN priors. This is particularly important when gradient signals are weak in methods such as SCRUB. In contrast, inverting gradients succeeds with larger unlearned batches and leaks different information across runs. Their success against SOTA unlearning models highlight potential privacy risks.

Our investigations led to developing SGRU, which removes forget set influence by targeting its subspace. This novel approach improves robustness against gradient-based attacks while maintaining stable, controllable unlearning. Compared to methods like SCRUB, which oscillates and require careful tuning, SGRU offers smoother performance and better retention of useful knowledge. SGRU also shows robustness to current attacks, but further targeted attacks and broader evaluations are needed to confirm its generality.

7.2 Future work

We plan to integrate Optuna for hyperparameter tuning to enhance user experience and model performance. The process involves defining an objective function with hyperparameters, training the model, and optimizing across multiple trials. Unlike standard training, in unlearning the objective is to approximate a model trained only on retain data rather than simply minimizing classification loss. However, in a realistic setting we do not have access to this model. Key design choices include determining the unlearning depth and balancing forget set performance against test set alignment.

We also plan to extend support for more datasets used in machine unlearning research, such as MNIST, UTKFace and CelebA. Additionally, we plan to add other existing adversarial attacks as outlined in [12], which will help to evaluate diverse unlearning scenarios, such as retain-set-based methods, which remain challenging for reconstruction attacks.

With the rapid advancement in large language models (LLMs), we aim to extend current work to cover this domain. Future work will involve extending current attacks and as well as LLM-specific unlearning and attack methods, expanding on the work covered in existing pipelines such as [20][21].

References

- [1] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020.
- [2] A. Mantelero, “The eu proposal for a general data protection regulation and the roots of the ‘right to be forgotten’,” *Computer Law Security Review*, vol. 29, no. 3, pp. 229–235, 2013.
- [3] Z. Izzo, M. A. Smart, K. Chaudhuri, and J. Zou, “Approximate data deletion from machine learning models,” in *International Conference on Artificial Intelligence and Statistics*, pp. 2008–2016, PMLR, 2021.
- [4] H. Xu, T. Zhu, L. Zhang, W. Zhou, and P. S. Yu, “Machine unlearning: A survey,” *arXiv preprint arXiv:2306.03558*, 2023.
- [5] A. Golatkar, A. Achille, and S. Soatto, “Eternal sunshine of the spotless net: Selective forgetting in deep networks,” in *Proceedings of the 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9301–9309, IEEE, 2020.
- [6] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership inference attacks against machine learning models,” in *2017 IEEE symposium on security and privacy (SP)*, pp. 3–18, IEEE, 2017.
- [7] L. Zhu, Z. Liu, and S. Han, *Deep leakage from gradients*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [8] X. Yuan, P. He, Q. Zhu, and X. Li, “Adversarial examples: Attacks and defenses for deep learning,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 30, no. 9, pp. 2805–2824, 2019.
- [9] W. Wang, Z. Tian, C. Zhang, and S. Yu, “Machine unlearning: A comprehensive survey,” *arXiv preprint arXiv:2405.07406*, 2024.
- [10] X. F. Cadet, A. Borovykh, M. Malekzadeh, S. Ahmadi-Abhari, and H. Haddadi, “Deep unlearn: Benchmarking machine unlearning,” *arXiv preprint arXiv:2410.01276*, 2024.
- [11] W. Shi, A. Ajith, M. Xia, Y. Huang, D. Liu, T. Blevins, D. Chen, and L. Zettlemoyer, “Detecting pretraining data from large language models,” 2023.
- [12] T. T. Nguyen, T. T. Huynh, Z. Ren, P. L. Nguyen, A. W.-C. Liew, H. Yin, and Q. V. H. Nguyen, “A survey of machine unlearning,” 2024.
- [13] H. Hu, S. Wang, T. Dong, and M. Xue, “Learn What You Want to Unlearn: Unlearning Inversion Attacks against Machine Unlearning,” in *2024 IEEE Symposium on Security and Privacy (SP)*, (Los Alamitos, CA, USA), pp. 3257–3275, IEEE Computer Society, May 2024.
- [14] M. Kurmanji, P. Triantafillou, J. Hayes, and E. Triantafillou, “Towards unbounded machine unlearning,” *Advances in neural information processing systems*, vol. 36, pp. 1957–1987, 2023.
- [15] S. Goel, A. Prabhu, A. Sanyal, S.-N. Lim, P. Torr, and P. Kumaraguru, “Towards adversarial evaluations for inexact machine unlearning,” *arXiv preprint arXiv:2201.06640*, 2022.
- [16] Z. Li, J. Zhang, L. Liu, and J. Liu, “Auditing privacy defenses in federated learning via generative gradient leakage,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10132–10142, 2022.
- [17] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, “Inverting gradients - how easy is it to break privacy in federated learning?,” *Advances in neural information processing systems*, vol. 33, pp. 16937–16947, 2020.

- [18] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramèr, “Membership inference attacks from first principles,” in *2022 IEEE Symposium on Security and Privacy (SP)*, p. 1897–1914, IEEE, May 2022.
- [19] T. Leemann, M. Pawelczyk, and G. Kasneci, “Gaussian membership inference privacy,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 73866–73878, 2023.
- [20] V. Dorna, A. Mekala, W. Zhao, A. McCallum, J. Z. Kolter, and P. Maini, “OpenUnlearning: A unified framework for llm unlearning benchmarks.” <https://github.com/locuslab/open-unlearning>, 2025. Accessed: February 27, 2025.
- [21] W. Shi, J. Lee, Y. Huang, S. Malladi, J. Zhao, A. Holtzman, D. Liu, L. Zettlemoyer, N. A. Smith, and C. Zhang, “Muse: Machine unlearning six-way evaluation for language models,” 2024.